

🚩 AI-POWERED CTF SOLVER

Full System Architecture & Design Document

Multi-Platform · Parallel Execution · Kali-Native · LLM-Orchestrated

Remote Kali VM

Web MCP

Claude AI Core

v1.0 — March 2026

Table of Contents

- 1** System Overview & Goals 3
- 2** High-Level Architecture 4
- 3** Component Deep-Dives 5
 - 3.1 Web MCP Layer 5
 - 3.2 Claude AI Orchestrator 6
 - 3.3 Challenge Classifier 7
 - 3.4 Remote Kali Execution Engine 8
 - 3.5 Parallel Worker Pool 9
 - 3.6 Flag Submission Module 9
- 4** Challenge Type Playbooks 10
- 5** Data Flow & Agent Loop 12
- 6** Remote Kali VPS Setup 13
- 7** Security Considerations 14
- 8** Tech Stack Summary 15

1. System Overview & Goals

This document describes the complete architecture of an AI-powered Capture The Flag (CTF) solver — an autonomous agent system capable of reading challenges from any CTF platform, reasoning about solution strategies, dispatching tool use to a remote Kali Linux environment, and submitting flags, all with minimal human intervention.

Core Design Goals

- › Platform-agnostic: works with CTFd, PicoCTF, HackTheBox, Pwn.College, and any HTTP-based CTF via Web MCP
- › Parallel execution: multiple challenges solved concurrently via a worker pool
- › Kali-native tooling: all standard security tools available with no restrictions on the execution VM
- › LLM-orchestrated: Claude as the reasoning brain — classifies challenges, selects tools, interprets outputs, iterates
- › Extensible: new challenge types and tool integrations added via playbook plugins

Key Design Philosophy

The AI does not need to know how to use every tool — it needs to reason about WHICH tool to try, pass

the right arguments, interpret the output, and decide the next step. The Kali VM handles execution.

Claude's role is strategic planning and synthesis, not low-level tool invocation.

2. High-Level Architecture

The system is composed of five primary layers that communicate via structured message passing:

LAYER 1 — INPUT / INGESTION › Web MCP Agent
Reads CTF platform pages, extracts challenge metadata, downloads attachments, stores locally
LAYER 2 — REASONING / ORCHESTRATION › Claude AI Core
Classifies challenge type, selects strategy playbook, plans tool invocations, synthesizes results
LAYER 3 — EXECUTION › Remote Kali VM via SSH/API
Runs tools: binwalk, pwntools, hashcat, gdb, sqlmap, ffuf, john, etc. Full root access
LAYER 4 — COORDINATION › Parallel Worker Pool
Manages N concurrent challenge workers, handles queuing, rate limits, retry logic
LAYER 5 — OUTPUT › Flag Submitter
Validates flag format, submits via platform API or Web MCP, logs results

All layers are coordinated by the Claude AI Core, which maintains a per-challenge state machine and issues structured tool calls that are routed to the appropriate downstream component.

3. Component Deep-Dives

3.1 Web MCP Layer

The Web MCP (Model Context Protocol) layer acts as the eyes of the system. It provides Claude with the ability to interact with CTF platform web interfaces as if browsing a site.

Responsibilities

- › Authenticate to CTF platforms using stored session cookies or credentials
- › Navigate challenge listings and extract: title, description, category, point value, hints, file attachments
- › Download challenge binary/archive files to a shared working directory on the Kali VPS
- › Submit flags via platform HTTP API endpoints or form submissions
- › Detect flag format from platform config (e.g. flag{...}, CTF{...}, picoCTF{...})

Generic Platform Adapter

For platforms without a dedicated MCP tool, a generic adapter uses:

- › Playwright-based headless browser for JavaScript-heavy platforms
- › requests + BeautifulSoup for simple HTML CTFd-style platforms
- › OpenAPI / REST client when the platform exposes a documented API (e.g. HTB)

Platform Support Matrix

CTFd (self-hosted)	→ REST API + HTML scraping
PicoCTF	→ HTML scraping + file download
HackTheBox	→ Official HTB API (v4)
Pwn.College	→ Playwright headless
Generic	→ Auto-detect via DOM structure heuristics

3.2 Claude AI Orchestrator

Claude (claude-sonnet-4) serves as the central reasoning engine. It operates in an agentic loop: receive challenge context, reason about approach, call tools, observe results, update state, repeat until flag found or timeout.

System Prompt Design

The orchestrator system prompt instructs Claude to:

- › Think step-by-step about challenge type before issuing any tool calls
- › Output structured JSON for all tool invocations (tool name, args, expected output format)
- › Maintain a scratchpad of observations and hypotheses across iterations
- › Recognize flag patterns in raw tool output automatically
- › Know when to escalate difficulty (try harder techniques) vs. abandon and note unsolved

```

SYSTEM: You are a CTF solver agent. For each challenge you receive:
  1. Identify challenge type from category + description + file extensions
  2. Select appropriate tools from the playbook
  3. Issue tool calls as JSON: {tool, args, rationale}
  4. Interpret tool output and extract flag if present
  5. If no flag, revise hypothesis and retry with different approach
  6. Return: {flag | null, steps_taken, confidence}

```

3.3 Challenge Classifier

Before the main solving loop, a lightweight classification pass assigns each challenge to one or more categories. This gates which playbook(s) are loaded into Claude's context.

Category	Signal Keywords / File Types	Primary Tools
Cryptography	encrypt, cipher, RSA, XOR, .enc, .key, base64	openssl, python-crypto, RsaCtfTool, xortool
Binary Exploitation	pwn, overflow, shellcode, .elf, ret2libc, heap	pwntools, gdb-peda, ROPgadget, checksec
Reverse Engineering	reverse, binary, .exe, .elf, .apk, disasm	ghidra, radare2, angr, strings, ltrace
Web Exploitation	SQL, XSS, SSRF, LFI, cookie, .php, .js	sqlmap, ffuf, burpsuite, curl, nikto
Forensics	steg, .png, .jpg, .pcap, .zip, metadata	binwalk, steghide, volatility, wireshark
OSINT	find, locate, person, username, social	theHarvester, sherlock, exiftool, curl
Misc / Jail	python jail, bash restricted, sandbox escape	Python one-liners, bash tricks, env inspection

3.4 Remote Kali Execution Engine

All tool execution happens on a remote Kali Linux VPS. Claude communicates with it via a lightweight execution API — a thin HTTP server running on the Kali box that accepts tool invocations and returns stdout/stderr.

Execution API Design

```

POST /exec
{ "tool": "binwalk", "args": ["-e", "/work/challenge.png"], "timeout": 30 }

Response:
{ "stdout": "...", "stderr": "", "exit_code": 0, "duration_ms": 412 }

```

File Workspace

- › Each challenge gets an isolated `/work/<challenge_id>/` directory on the Kali box
- › Files downloaded by Web MCP are placed here before tool execution begins
- › Tools write output artifacts (extracted files, decrypted content) back to this directory
- › Claude can request directory listings to discover new artifacts mid-solve

Security Note

The Kali execution API should be bound to localhost or a private VPN interface only.
Use a dedicated non-root challenge user for tool execution (sudo allowed for specific tools).
Network access from within the Kali box should be restricted to CTF infrastructure only.

3.5 Parallel Worker Pool

The worker pool manages concurrent challenge solving. Each worker is an independent Claude agent context that operates on a single challenge. Workers communicate only through a shared result store.

- › Default pool size: 4–8 workers (limited by API rate limits and Kali resource constraints)
- › Work queue: challenges sorted by estimated difficulty (points ascending for quick wins first)
- › Each worker has a configurable `max_iterations` (default: 15) and `timeout` (default: 10 min)
- › Solved flags are immediately submitted without waiting for other workers
- › Workers can request additional Kali compute for heavy tasks (hashcat GPU cracking)

3.6 Flag Submission Module

Once a flag is extracted, the submission module validates format, deduplicates, and submits via the platform's API or the Web MCP layer. Results are logged with the full solve chain for post-event review.

4. Challenge Type Playbooks

Each playbook is a structured prompt template loaded into Claude's context alongside the challenge. It contains: recommended tool sequence, common failure modes, flag pattern hints, and escalation paths.

Cryptography Playbook

- › Step 1: Identify algorithm — frequency analysis, ciphertext length, file magic bytes
- › Step 2: Try common encodings first — base64, hex, rot13, base85
- › Step 3: Classic ciphers — Caesar, Vigenere, substitution (use dcode.fr heuristics)
- › Step 4: Modern crypto — RSA (check for weak e, small N, common factor attacks via RsaCtfTool)
- › Step 5: XOR analysis — xortool for repeating key detection
- › Escalation: custom cipher analysis, sage math scripts, lattice attacks

Binary Exploitation Playbook

- › Step 1: checksec — determine protections (NX, PIE, canary, RELRO)
- › Step 2: file + strings — quick recon on binary type and hardcoded values
- › Step 3: Run normally — observe crash behaviour, error messages
- › Step 4: pwntools script — fuzz input size, identify overflow offset with cyclic()
- › Step 5: Determine exploit class — ret2win, ret2libc, rop chain, format string, heap
- › Step 6: Build and test exploit locally, then remote
- › Escalation: GDB scripting, one_gadget, patchelf for libc version matching

Reverse Engineering Playbook

- › Step 1: file + strings — get quick wins from hardcoded flags or keys
- › Step 2: ltrace / strace — observe library calls and comparisons at runtime
- › Step 3: Ghidra / radare2 — decompile main(), look for strcmp/memcmp on input
- › Step 4: angr symbolic execution — automatically find satisfying inputs
- › Step 5: Dynamic analysis — patch jumps in GDB to bypass checks
- › Escalation: anti-debug bypass, custom VM reversing, obfuscated bytecode

Web Exploitation Playbook

- › Step 1: Manual recon — view source, check /robots.txt, /.git/, /admin
- › Step 2: Parameter fuzzing — ffuf on paths and parameters
- › Step 3: Injection testing — sqlmap for SQLi, XSS payload lists
- › Step 4: Auth flaws — JWT manipulation, cookie tampering, IDOR testing
- › Step 5: SSRF / LFI — probe internal services, /etc/passwd, /proc/self
- › Escalation: deserialization, SSTI, prototype pollution, race conditions

Forensics Playbook

- › Step 1: file + exiftool — identify actual file type, extract metadata
- › Step 2: binwalk -e — extract embedded files from binary blobs or images
- › Step 3: strings + grep — search for flag pattern in raw bytes
- › Step 4: Steganography — steghide, zsteg (PNG), stegsolve LSB analysis
- › Step 5: PCAP analysis — wireshark/tshark: follow TCP streams, extract files, find creds
- › Step 6: Memory forensics — volatility: process list, network connections, strings

5. Data Flow & Agent Loop

The following describes one complete challenge-solve cycle from challenge discovery to flag submission:

Step 01 **DISCOVER**

Web MCP scans platform. Extracts all unsolved challenges as structured JSON: {id, title, category, description, hints[], files[]}

Step 02 **QUEUE**

Worker pool receives challenge list. Sorted by points (ascending). N workers each claim one challenge.

Step 03 **FETCH FILES**

Worker MCP downloads all file attachments to /work/<id>/ on Kali VPS. Computes file hashes for dedup.

Step 04 **CLASSIFY**

Claude reads challenge metadata + file types. Assigns category (may be multi-label). Loads appropriate playbook.

Step 05 **PLAN**

Claude outputs a structured solve plan: ordered list of tool calls with rationale. This is the initial hypothesis.

Step 06 **EXECUTE**

Each tool call is sent to Kali Execution API. stdout/stderr returned to Claude. Max 30s per call by default.

Step 07 **OBSERVE**

Claude reads tool output. Checks for flag pattern match. Updates scratchpad with observations.

Step 08 **ITERATE**

If no flag: Claude updates hypothesis, selects next tool/approach. Repeats from step 06 up to max_iterations.

Step 09 **EXTRACT**

Flag found in tool output. Claude validates against known format regex. Returns {flag, confidence, solve_chain}.

Step 10 **SUBMIT**

Flag submitted via platform API. Response confirmed. Worker marked complete. Next challenge claimed.

6. Remote Kali VPS Setup

Recommended VPS Specs

Resource	Minimum	Recommended
CPU	2 vCPUs	4–8 vCPUs (for parallel workers)
RAM	4 GB	8–16 GB (for Ghidra, angr, volatility)
Storage	40 GB SSD	100 GB SSD (capture file archives)
GPU	None	Optional: NVIDIA for hashcat acceleration
OS	Kali Linux 2024.x	Kali Linux 2025.x (latest rolling)
Network	100 Mbps	1 Gbps (fast file downloads from CTF servers)

Execution API Server

A lightweight FastAPI server runs on the Kali box, exposing tool execution endpoints. It handles:

- › Auth via API key header (X-CTF-Key)
- › Sandboxing via subprocess with timeout and resource limits (ulimit)
- › File access scoped to /work/<challenge_id>/ per request
- › Structured stdout/stderr/exit_code responses

```
# Install on Kali VPS:
pip install fastapi uvicorn
uvicorn ctf_exec_api:app --host 0.0.0.0 --port 8765 --workers 4
```

Key Tools Pre-Installed

- › Crypto: openssl, RsaCtfTool, xortool, sage, hashcat, john
- › Pwn: pwntools, gdb + peda/pwndbg, ROPgadget, one_gadget, patchelf
- › Rev: ghidra, radare2, angr, binwalk, apktool, jadx
- › Web: sqlmap, ffuf, nikto, curl, httpie, burpsuite-headless
- › Forensics: steghide, zsteg, stegsolve, volatility3, Wireshark, foremost
- › OSINT: theHarvester, sherlock, exiftool, whois, nmap

7. Security Considerations

Critical: Isolate the Execution Environment

The Kali VPS has full tool access by design — this is intentional. However, it **MUST** be isolated from production infrastructure. Treat it as a throwaway research VM.

Network Isolation

- › Bind the Execution API to a private interface (e.g. WireGuard VPN) — never expose publicly
- › Use a dedicated VPS not shared with other services
- › Firewall rules: allow only VPN ingress to port 8765, block all other inbound
- › Egress: restrict to CTF platform IPs + package repos during competitions

API Key & Auth

- › Execution API protected with a 256-bit random API key stored in orchestrator secrets
- › All traffic over VPN tunnel (WireGuard) or SSH tunnel — no plaintext
- › Rotate API key after each competition

Challenge File Hygiene

- › All downloaded challenge files treated as adversarial (malware CTFs exist)
- › Never execute unknown binaries outside the Kali sandbox
- › Use dedicated challenge user (ctf_runner) with no access to VPS system files
- › Automated cleanup: /work/ directories wiped after each competition

8. Tech Stack Summary

Component	Technology	Purpose
AI Core	Claude claude-sonnet-4 (Anthropic API)	Challenge reasoning, tool planning, flag extraction
Web MCP	MCP + Playwright / httpx	Platform interaction, file download, flag submission
Execution API	FastAPI + uvicorn (Python)	Thin HTTP wrapper over Kali tool execution
Worker Pool	Python asyncio + aiohttp	Parallel challenge workers, queue management
State Store	Redis (in-memory)	Challenge state, worker assignments, flag cache
Orchestrator	Python 3.12+	Top-level coordination, logging, reporting
VPS OS	Kali Linux Rolling	Full security toolchain, no restrictions
Tunnel	WireGuard VPN	Secure orchestrator ↔ Kali communication
Config	YAML + .env secrets	Platform credentials, API keys, competition config

Repository Structure

```
ctf-solver/
├── orchestrator/           # Main Python orchestrator
│   ├── main.py             # Entry point, worker pool
│   ├── claude_agent.py     # Claude API wrapper + agentic loop
│   ├── classifier.py       # Challenge type classifier
│   └── submitter.py        # Flag validation + submission
├── mcp/                   # Web MCP adapters
│   ├── generic.py          # Auto-detect platform adapter
│   ├── ctf.py             # CTFd REST API adapter
│   └── htb.py              # HackTheBox API v4 adapter
├── kali_api/              # Runs ON the Kali VPS
│   ├── server.py           # FastAPI execution server
│   └── tools.py            # Tool invocation helpers
├── playbooks/             # Per-category prompt templates
│   ├── crypto.md
│   ├── pwn.md
│   └── web.md              # etc.
└── config/
    ├── competition.yaml    # Platform URL, flag format, creds
    └── .env                # API keys (gitignored)
```

Next Steps

1. Set up Kali VPS and install `kali_api/server.py` with WireGuard tunnel
2. Configure `competition.yaml` for your target platform
3. Implement the MCP adapter for the specific CTF site
4. Run `orchestrator/main.py` — workers will auto-claim and start solving
5. Monitor the live dashboard for solved flags and unsolved challenge analysis